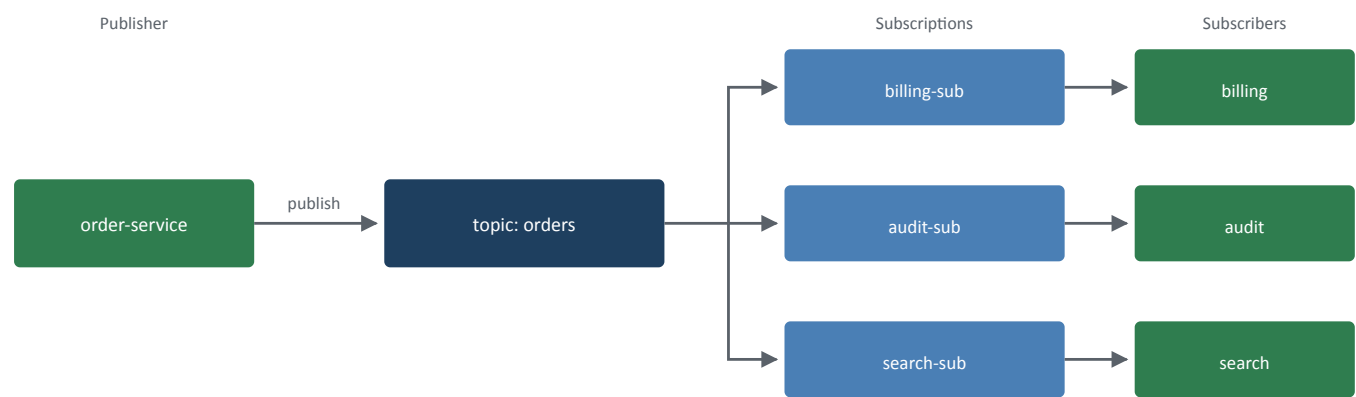


Pub/Sub

1. What is Pub/Sub?

Pub/Sub is a messaging pattern where a sender publishes a message to a topic, and the messaging system delivers it to every subscriber that registered an interest.

The sender and the receiver never refer to each other. The topic sits between them, which is the whole point of the pattern.



One published message is delivered once to each subscription. Adding a fourth subscriber changes nothing about the publisher.

2. Why use Pub/Sub?

Problem	What Pub/Sub does
A slow consumer holds up the caller	The publisher returns as soon as the message is stored
Adding a consumer means changing the producer	Add a subscription. The producer is untouched
A consumer being down loses the work	Messages are kept until they are acknowledged
A traffic spike overwhelms a service	The backlog absorbs it and the consumer catches up
Services know too much about each other	They share a message format and nothing else

The cost is that the answer arrives later, or not at all. A REST call returns a result; a published message returns only the fact that it was accepted. Anything that needs a reply needs a second mechanism.

3. Publisher

The publisher is the service that sends a message to a topic. It does not know who receives it, or whether anyone does.

```
Publisher publisher = Publisher.newBuilder(
    TopicName.of(projectId, "orders")).build();

PubsubMessage message = PubsubMessage.newBuilder()
    .setData(ByteString.copyFromUtf8(json))
    .putAttributes("eventType", "order.placed")
    .build();

ApiFuture<String> messageId = publisher.publish(message);
```

Publishing is fast because it does not wait for consumers. The call returns once the message is safely stored, which is unrelated to whether anything has processed it.

4. Subscriber

The subscriber is the service that receives messages from a subscription and processes them.

```
MessageReceiver receiver = (message, consumer) -> {
    process(message.getData().toStringUtf8());
    consumer.ack();
};

Subscriber subscriber = Subscriber.newBuilder(
    ProjectSubscriptionName.of(projectId, "billing-sub"), receiver).build();
subscriber.startAsync().awaitRunning();
```

That example is a pull subscriber. The other kind is push, and section 9 covers the difference.

5. Topic

A topic is the named channel a publisher sends to. It keeps no record of who is listening.

A topic with no subscriptions throws messages away. Nothing is stored for a consumer that does not exist yet, so the subscription has to be created before the first message is published, not after.

6. Subscription

A subscription is a named stream of messages from one topic to one consumer.

Each one keeps its own state, and that is what makes subscriptions independent of each other:

A subscription holds	Which means
Its own backlog	Messages waiting for this consumer only
Its own acknowledgement state	What this consumer has finished
Its own configuration	Deadline, retention, filter, dead letter topic

The consuming side creates the subscription. The team that wants the messages creates it against someone else's topic, and the publisher is neither involved nor informed.

7. Topic against subscription

	Topic	Subscription
Is	Where messages are published	Where messages are delivered from
Created by	The producing side	Each consuming side
Stores messages	No, it forwards	Yes, until acknowledged
Knows about	Nothing downstream	Exactly one topic
How many	One per kind of event	One per interested consumer

Two rules follow, and between them they explain every Pub/Sub topology.

One topic to many subscriptions is fan-out. Each subscription receives its own copy of every message, with its own backlog and its own acknowledgements. A consumer that is down or slow affects nobody else.

One subscription to many instances is load sharing. Each message goes to exactly one instance, so adding instances increases throughput without duplicating work.

```
orders (topic)
├─ billing-sub → billing, 3 instances    each message handled once
├─ audit-sub   → audit,   1 instance    each message handled once
└─ search-sub  → search,  2 instances    each message handled once
```

One publish produces three deliveries there, not six. The instance count never multiplies anything.

Getting this backwards is the classic mistake. Giving each instance of the billing service its own subscription makes all three process every message, so every card is charged three times.

8. Subscription lifecycle

Two settings decide how long a subscription and its messages survive, and both have deleted data for somebody.

Setting	Controls	Usual default
Message retention	How long unacknowledged messages are kept	7 days
Subscription expiration	How long an unused subscription survives	31 days

Message retention is the limit on how long a consumer can be down. Messages wait in the backlog until they are acknowledged or until retention runs out, and then they are gone. A consumer offline for longer than the retention period comes back to a gap, with nothing recording what it missed.

Subscription expiration removes the subscription itself. A subscription that nothing connects to for the expiry period is deleted, and its backlog with it. This is what happens to a subscription created ahead of a service that was never finished.

Both can be extended, and neither should be relied on. Retention can be raised, but a backlog measured in days means something is already wrong. The answer to a long outage is to fix the consumer, not to store more.

Retention also enables replay: with `seek`, a subscription can be moved back to a timestamp inside the retention window and receive those messages again.

9. Push against pull subscriptions

A subscription delivers one of two ways, and it is chosen when the subscription is created.

	Pull	Push
Who starts	The subscriber asks	Pub/Sub sends an HTTP request
Acknowledgement	An explicit <code>ack()</code> call	The HTTP status code
Needs	An outbound connection only	A public HTTPS endpoint
Flow control	The subscriber sets its own limits	Pub/Sub adjusts the rate
Throughput	High	Lower
Suits	Long-running services	Cloud Run, Cloud Functions, webhooks

In a push subscription the status code is the acknowledgement. There is no `ack()` to call: any 2xx means the message is done, and anything else means send it again.

```
{
  "message": {
    "data": "eyJvcmlrcklkIjo0MX0=",
    "messageId": "2070443601311540",
    "attributes": { "eventType": "order.placed" }
  },
  "subscription": "projects/acme/subscriptions/billing-sub"
}
```

```
@PostMapping("/pubsub/orders")
public ResponseEntity<Void> receive(@RequestBody PushEnvelope envelope) {
    charge(envelope.decodedData());
    return ResponseEntity.ok().build();    // this is the acknowledgement
}
```

The payload is base64 in that envelope, which catches people out the first time.

Push needs the endpoint reachable from Google. That means public HTTPS with a valid certificate, or a connector into the network. Pull only needs an outbound connection, which is why it works from anywhere.

Pull scales better because the subscriber sets the pace. It asks for what it can handle. With push, Pub/Sub decides the rate and backs off when the endpoint returns errors, so a slow consumer is discovered by failing rather than by declining work.

Push is the right answer for serverless. There is no long-running process to hold a streaming connection open, so an HTTP request that also starts the container is exactly what is needed.

10. Message and its attributes

A message is the unit of data: a payload of bytes, plus attributes, an id and a publish timestamp.

```
{
  "orderId": 41,
  "customerId": 9012,
  "amount": 250.00
}
```

Part	Holds
Data	The payload, as bytes. Usually JSON
Attributes	Key-value metadata, such as <code>eventType</code>
Message id	Assigned by Pub/Sub, unique per message
Publish time	When Pub/Sub accepted it
Ordering key	Optional. Messages with the same key keep their order

Put attributes to work. A subscription can filter on them, so a consumer that only wants `eventType=order.cancelled` never receives the rest.

Keep messages small. When the payload would be large, publish the id and let the consumer fetch the record. A message is a notification, not a transfer mechanism.

11. Message ordering

Pub/Sub does not guarantee order by default. Two messages published a millisecond apart can arrive in either sequence, because they may be handled by different servers and delivered to different instances.

An ordering key is how order is asked for. Messages published with the same key are delivered in publish order, one at a time, to one subscriber.

```
Publisher publisher = Publisher.newBuilder(topicName)
    .setEnableMessageOrdering(true)
    .build();

PubsubMessage message = PubsubMessage.newBuilder()
    .setData(ByteString.copyFromUtf8(json))
    .setOrderingKey("order-41")
    .build();
```

The subscription has to allow it too, so ordering is a decision both sides make.

	Without an ordering key	With one
Delivery order	Any	Publish order, within the key
Scope	—	Per key, never global
Parallelism	Full	One message at a time per key
A failed message	Blocks nothing	Blocks later messages with that key

The blocking is the part that surprises people. If `order-41` fails and is retried, every later message for `order-41` waits behind it. That is the correct behavior, and it means a poison message stops one key's stream until it succeeds or is dead-lettered.

Prefer designing order out. Putting a version or a timestamp in the message and ignoring anything older than what has already been applied gives the same result with none of the cost. Ordering keys are for the cases where that is genuinely impossible.

12. Acknowledgement and the ack deadline

An acknowledgement tells Pub/Sub that a message was processed and can be dropped.

```
consumer.ack();    // done, do not send it again
consumer.nack();   // failed, send it again now
```

Acknowledge after successful processing, never on receipt. This is the rule the whole section rests on. Acknowledging when the message arrives means a crash half way through loses the work, because Pub/Sub has already been told it is finished.

```
MessageReceiver receiver = (message, consumer) -> {  
    try {  
        charge(message);           // do the work first  
        consumer.ack();             // then say it is done  
    } catch (Exception e) {  
        consumer.nack();            // ask for it again  
    }  
};
```

The acknowledgement deadline is how long Pub/Sub waits. It starts when the message is delivered, and if nothing has arrived by the time it passes, the message is sent again.

	Typical value
Default deadline	10 seconds
Maximum	600 seconds
Extended while working	The client library does it automatically

The client library extends the deadline in the background for as long as the handler is running, so a slow handler does not normally cause redelivery. A handler that blocks the library, or a process that stops responding, does.

What happens when an acknowledgement does not arrive is the next section.

13. Retry and redelivery

A message is redelivered whenever Pub/Sub has not been told it is finished. The reasons divide into two kinds.

Acknowledgement failure: the work succeeded and the acknowledgement did not arrive. The consumer crashed after acting, or the network dropped the reply, or the deadline passed a moment before it was sent.

Processing failure: the work did not succeed. The handler threw, or it called `nack` deliberately.

Cause	Kind	Was the work done?
The deadline passed	Either	Possibly
<code>nack</code>	Processing	No
The consumer crashed mid-handler	Processing	Partly
The consumer crashed after the work	Acknowledgement	Yes
The network dropped the acknowledgement	Acknowledgement	Yes

The consumer cannot tell which happened. A redelivered message looks the same in every row of that table, which is the whole reason the next section exists.

A retry policy decides how long Pub/Sub waits before trying again. The default is to redeliver immediately; the alternative is exponential backoff.

Setting	Means	Usual default
Minimum backoff	The wait before the first retry	10 seconds
Maximum backoff	The ceiling the wait grows to	600 seconds

Exponential backoff doubles the wait each time, from the minimum toward the maximum:

```
attempt 1    wait 10s
attempt 2    wait 20s
attempt 3    wait 40s
attempt 4    wait 80s
...          doubling until the maximum, then holding there
```

Backoff exists to protect whatever the consumer depends on. A message failing because the database is down, retried immediately and forever, keeps the database down. Waiting longer each time gives it room to recover, which is the difference between a slow recovery and no recovery.

14. What is a dead-letter topic, and why use it?

A dead-letter topic is a separate topic where messages are sent after a set number of failed delivery attempts.

It is configured on the subscription, with two values: the topic to send to, and how many attempts to allow first.

Without one, a message that can never succeed is retried forever. It is never lost and never finished, and it consumes a share of the consumer indefinitely.

Why it is used	Detail
Stops infinite retries	The message leaves the subscription after N attempts
Keeps the message	It is moved aside rather than dropped
Keeps the rest flowing	One bad message stops blocking the ordering key
Gives somewhere to alert	A non-empty dead-letter backlog needs a person

The message arrives carrying attributes that say which subscription gave up on it and how many attempts were made, which is usually enough to start investigating without reproducing anything.

A dead-letter topic needs its own subscription. Without one it is a topic with no subscribers, and by section 5 that means the messages are discarded on arrival. The most common mistake here is configuring the dead-letter topic, feeling safe, and quietly throwing away every failure.

Alert on its backlog, not on its rate. Any message arriving there is a message no amount of retrying fixed, so the useful alert is that the count is above zero.

15. Concurrency and parallel message processing

A subscriber handles many messages at the same time rather than one after another. There are two independent ways to get that.

Lever	Adds parallelism by
More instances	Running more copies of the service on one subscription
More threads per instance	Running the handler on many threads inside one process

Both are in play at once, so eight threads across ten instances is eighty messages being processed simultaneously.

```
Subscriber subscriber = Subscriber.newBuilder(subscription, receiver)
    .setParallelPullCount(2)
    .setExecutorProvider(InstantiatingExecutorProvider.newBuilder()
        .setExecutorThreadCount(8)
        .build())
    .setFlowControlSettings(FlowControlSettings.newBuilder()
        .setMaxOutstandingElementCount(1_000L)
        .build())
    .build();
```

Setting	Controls
Executor thread count	How many messages one instance handles at once
Max outstanding elements	How many delivered but unacknowledged messages it holds
Max outstanding bytes	The same limit, measured in memory
Parallel pull count	How many streaming connections it opens

The handler runs on many threads, so it has to be thread-safe. Shared mutable state in a receiver is a bug that only appears under load.

Flow control is what stops a backlog becoming an out-of-memory error. A consumer restarting against a large backlog will take messages as fast as it can unless a limit says otherwise, and the first symptom is the process being killed.

Concurrency multiplies the load on everything downstream. Eighty concurrent handlers means eighty concurrent database connections, and the pool is usually much smaller than that. The consumer is rarely the thing that breaks first.

Saturation and the acknowledgement deadline make a feedback loop. When every thread is busy, messages wait, their deadlines expire, and they are redelivered, which adds more work to an already saturated consumer. Flow control set below what the downstream can take is what prevents it.

16. Message ordering with concurrency

Ordering and concurrency pull in opposite directions, and Pub/Sub reconciles them one key at a time.

Situation	What happens
No ordering key	Full parallelism, any order
One key, many messages	Serial. One at a time, in publish order
Many keys	Parallel across keys, serial within each
More threads than keys	The extra threads sit idle

Parallelism is bounded by the number of distinct keys, not by the thread count. This is the sentence worth remembering, because it means the ordering key is a throughput decision as much as a correctness one.

key = orderId	thousands of keys	near-full parallelism
key = customerId	hundreds of keys	good, if per-customer order is enough
key = region	five keys	five messages at a time, whatever the thread count
key = a constant	one key	fully serial. The system is now single-threaded

Choose the most granular key that is still correct. If the requirement is that events for one order are applied in order, the key is the order id. Using anything coarser serializes work that never needed to be.

A failing message blocks only its own key. With per-order keys, one poison message stalls one order while every other order keeps moving. With a coarse key it stalls everything sharing that key, which is how a single bad message stops a whole region.

That is the same trade as section 11, seen from the other side: ordering buys sequence and pays in parallelism, and the key decides how much.

17. What is idempotency, and why do Pub/Sub consumers need it?

An idempotent operation produces the same result whether it runs once or many times.

charge the card for order 41	not idempotent, twice means charged twice
set order 41's status to CHARGED	idempotent, twice means the same

Pub/Sub consumers need it because delivery is at least once. A message can arrive twice for reasons that are nobody's fault: the work succeeded and the acknowledgement was lost, the deadline passed while the handler was still running, or the consumer crashed after acting and before acknowledging.

Duplicates are not an edge case to guard against. At any real volume they are a certainty, so the consumer has to be written for them.

An idempotency key is the value that identifies the unit of work. A repeat is recognized by having seen that key before.

Key	Good for
The Pub/Sub message id	Catching a redelivery of the same message
A business key, such as the order id	Also catching the same event published twice

The business key is the stronger choice. A publisher that retries produces two messages with different ids and the same meaning, and only a business key notices.

```
@Transactional
public void handle(OrderPlaced event) {
    if (processed.existsById(event.orderId())) {
        return;
    }
    charges.save(new Charge(event.orderId(), event.amount()));
    processed.save(new ProcessedOrder(event.orderId()));
}
```

The check and the work must be in one transaction. Written separately, a crash between them leaves the work done and the key unrecorded, which is exactly the case this code exists to prevent.

A unique constraint is stronger than a check. Letting the database reject the second insert removes the race between two instances processing duplicates at the same moment, which the read-then-write version above still has.

18. Pub/Sub vs REST API

These are not competing technologies; most systems use both.

	REST API	Pub/Sub
Caller waits	Yes, for the response	No. There is no response
Knows the receiver	Yes, by address	No
Receivers per call	One	Every subscription
If the receiver is down	The call fails	The message waits
Adding a receiver	Change the caller	Add a subscription

Use REST when the answer is needed to continue. Reading a customer, checking a balance, validating a request. The caller cannot proceed without the response.

Use Pub/Sub when the work can happen afterwards. An order was placed, and billing, audit and search all need to know. None of them affects the reply the customer is waiting for.

19. Synchronous vs asynchronous communication

	Synchronous	Asynchronous
The caller	Waits for the response	Continues immediately
Failure appears	At once, to the caller	Later, to the consumer
Availability	The caller is down if the callee is	Independent
Latency	The sum of every call in the chain	The first step only
Debugging	A stack trace	Correlating across services

Synchronous calls couple availability. A chain of five services each at 99.9% is available 99.5% of the time, because every one of them has to be up at the same moment. Making a step asynchronous removes it from that calculation.

The cost is that nothing is immediately consistent. After the order is published, there is a period where the order exists and the search index does not know about it. That has to be acceptable to the product, not just to the architecture.

20. Point-to-point vs publish-subscribe

	Point-to-point	Publish-subscribe
Also called	A queue	A topic
Each message goes to	Exactly one consumer	Every subscriber
Purpose	Sharing out work	Announcing that something happened
Adding a consumer	The work is split further	The message is delivered again
Example	Resize these images	An order was placed

Pub/Sub does both, and the subscription is the switch. Many instances on one subscription is point-to-point, because each message is handled once. Many subscriptions on one topic is publish-subscribe, because each gets a copy.

That is the same distinction as section 6, and it is worth being able to state it either way round.

The pieces together

Piece	What it is
Topic	Where messages are published
Subscription	A stream of copies, one per interested consumer
Publisher	Sends, and does not know who receives
Subscriber	Receives from a subscription and processes
Message	Payload, attributes, id and timestamp
Acknowledgement	Proof it was handled, or it comes back
At least once	Why consumers must be idempotent
Ack deadline	How long before it comes back
Ordering key	How order is asked for, and what it costs
Backoff	Why a failing consumer does not hammer its database
Push or pull	Who starts the delivery, and what acknowledges it
Flow control	Why a backlog does not become an out-of-memory error

Piece	What it is
Retention	How long a consumer may be down
Idempotency key	How a repeat is recognized
Dead letter topic	Where a message that always fails ends up

For example, an order is placed. The order service writes it to its database and publishes one message to the `orders` topic, then returns to the customer. Three subscriptions each receive a copy: billing charges the card, audit records the event, and search updates its index. The search service is redeploying and its messages wait in the backlog until it returns. Billing processes the same message twice because an acknowledgement was lost, and charges once because it records message ids. Adding a fourth consumer later is a new subscription, and the order service never changes.

The idea worth carrying is that a subscription, not a subscriber, is the unit of delivery. Almost every Pub/Sub question turns out to be a question about how many subscriptions there are.